

High-Performance Implementation of Multigrid Solver for Lattice QCD

面向 Lattice QCD 传播子求解的高性能多重网格实现

汇报人 张鑫

中国科学院近代物理研究所

2026-10-11 15:05–15:20

项目 [https:](https://github.com/zhangxin8069/PyQCU)

[//github.com/zhangxin8069/PyQCU](https://github.com/zhangxin8069/PyQCU)

镜像 [https:](https://gitee.com/zhangxin8069/PyQCU)

[//gitee.com/zhangxin8069/PyQCU](https://gitee.com/zhangxin8069/PyQCU)

文档 [https://gitee.com/
zhangxin8069/PyQCU/tree/main/docs](https://gitee.com/zhangxin8069/PyQCU/tree/main/docs)

日志 [https:
//gitee.com/zhangxin8069/PyQCU/tags](https://gitee.com/zhangxin8069/PyQCU/tags)

性能结论统一采用 `git tag test27`: 66 个精确单元、132 条 side record

Wilson & Clover: fine-level Dirac operator conventions

Wilson Dslash

$$\begin{aligned}(H\psi)_x &= \sum_{\mu=0}^3 \left[(1 - \gamma_\mu) U_{x,\mu} \psi_{x+\hat{\mu}} \right. \\ &\quad \left. + (1 + \gamma_\mu) U_{x-\hat{\mu},\mu}^\dagger \psi_{x-\hat{\mu}} \right], \\ D_W &= (m_0 + 4)I - \frac{1}{2}H \\ &= (m_0 + 4)(I - \kappa H), \\ \kappa &= \frac{1}{2m_0 + 8}.\end{aligned}\tag{1}$$

Clover 改进 (当前 $u_0 = 1, c_{\text{sw}} = 1$)

$$\begin{aligned}Q_{\mu\nu} &= P_{\mu\nu} + P_{\nu,-\mu} + P_{-\mu,-\nu} + P_{-\nu,\mu} \\ C_{\mu\nu} &= Q_{\mu\nu} - Q_{\mu\nu}^\dagger \\ T_\rho &= -\frac{\kappa c_{\text{sw}}}{8u_0} \sum_{\mu < \nu} \gamma_\mu \gamma_\nu C_{\mu\nu} \\ A_\rho &= I_\rho + T_\rho\end{aligned}\tag{2}$$

Raw Dslash: eight-neighbor matrix-free stencil

Kernel flow / entry semantics

- Per site: four forward and four backward neighbors;
- $1 \pm \gamma_\mu$: four-spinor $\rightarrow$ two components;
- $SU(3)$ transport and eight-direction accumulation。

原始入口返回裸核 H ; 带单位项入口返回 $I - (\kappa/u_0)H$ 。

邻居旋量读取



$1 \pm \gamma_\mu$ 投影



$SU(3)$ 平移



八方向累加

性能本质: 最近邻模板天然 bandwidth-bound; $4 \rightarrow 2$ spin projection、 $SU(3)$ 重用和尾置时空维决定有效带宽。

Even-odd Schur: half-size outer Krylov problem

Block matrix and Schur complement

$$D_{\text{pc}} = \begin{pmatrix} A_e & -\kappa H_{eo} \\ -\kappa H_{oe} & A_o \end{pmatrix} \quad (3)$$

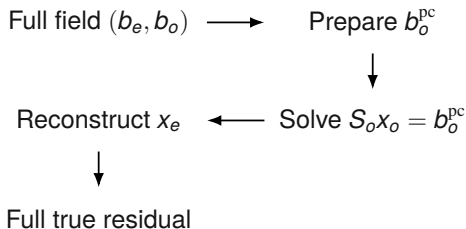
$$S_o = A_o - \kappa^2 H_{oe} A_e^{-1} H_{eo} \quad (4)$$

Solve and reconstruct

$$b_o^{\text{pc}} = b_o + \kappa H_{oe} A_e^{-1} b_e \quad (5)$$

$$S_o x_o = b_o^{\text{pc}} \quad (6)$$

$$x_e = A_e^{-1} (b_e + \kappa H_{eo} x_o) \quad (7)$$

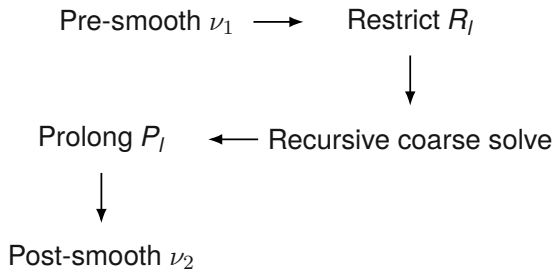


MultiGrid: null-space coarse space + smoother

Four core objects

- 近零模: $A_I B_I \approx 0$
- 局部正交: $P_I^\dagger P_I = I$
- 转移对: $R_I = P_I^\dagger$
- Galerkin 粗算子: $A_{I+1} = R_I A_I P_I$

$$e = e_{\text{high}} + e_{\text{low}}, \quad A_I B_I \approx 0 \quad (8)$$



Null-vector construction: inexact inverse correction + local QR

Algorithm 1 Null-vector construction

Input : $A_I, e_I, n_v, b, n_{\text{iter}}=2, \varepsilon_{\text{rtol}}=10^{-2}, \text{seed}=42$

for $j = 1, \dots, n_v$

$v_j \sim \mathcal{N}(0, I)$

repeat n_{iter}

$x_j \approx \text{solve}_{\text{rel}}(A_I x = A_I v_j; \varepsilon_{\text{rtol}})$

$v_j \leftarrow v_j - x_j$

$v_j \leftarrow v_j / \|v_j\|_2$

for aggregate c

$V_c = [v_j]_{j \in c}; \quad Q_c R_c = \text{reducedQR}(V_c)$

$P_{I|c} \leftarrow Q_c; \quad \text{cache blocked } P_I, V_I$

$R_I \leftarrow P_I^\dagger; \quad D_{I+1} \leftarrow R_I D_I P_I$

$$x_j \approx A_I^{-1}(A_I v_j), \quad (9)$$

$$v_j \leftarrow v_j - x_j, \quad v_j \leftarrow v_j / \|v_j\|_2$$

$$V_c = P_c R_c, \quad E = 2n_v \quad (10)$$

$$P_c^\dagger P_c = I_E$$

$$(R_I v)_X = \sum_{x \in \mathcal{A}_X} V_X^\dagger v(x) \quad (11)$$

$$(P_I z)(x) = V_X z(X)$$

$$A_{I+1} = R_I A_I P_I, \quad R_I = P_I^\dagger \quad (12)$$

关键实现： 相对容差、独立 RNG seed、分块 batched solve; threaded path 支持 breakdown/NaN 重采样, batch path 当前没有逐向量重试。

路径区分： local_orthogonalize 使用 batched reduced QR; Strict/reference 的 aggregate CGS 是另一条实现, 不能混写。

Preconditioned Bi-CGStab for coarse solves

Algorithm 2 Preconditioned Bi-CGStab

$r^{(0)} = b - Ax^{(0)}$
 $\tilde{r} = r^{(0)}$
for $i = 1, 2, \dots$
 $\rho_{i-1} = \tilde{r}^\dagger r^{(i-1)}$; **if** $\rho_{i-1} = 0$ **fail**
 if $i = 1$ **then** $p^{(1)} = r^{(0)}$
 else
 $\beta_{i-1} = \frac{\alpha_{i-1}\rho_{i-1}}{\omega_{i-1}\rho_{i-2}}$
 $p^{(i)} = r^{(i-1)} + \beta_{i-1}(p^{(i-1)} - \omega_{i-1}v^{(i-1)})$
 solve $M\hat{p} = p^{(i)}$; $v^{(i)} = A\hat{p}$

$\alpha_i = \rho_{i-1} / (\tilde{r}^\dagger v^{(i)})$
 $s = r^{(i-1)} - \alpha_i v^{(i)}$
if $\|s\| / \|b\| \leq \varepsilon$ **then** $x^{(i)} = x^{(i-1)} + \alpha_i \hat{p}$; **stop**
solve $M\hat{s} = s$; $t = A\hat{s}$
 $\omega_i = t^\dagger s / (t^\dagger t)$
 $r^{(i)} = s - \omega_i t$
 $x^{(i)} = x^{(i-1)} + \alpha_i \hat{p} + \omega_i \hat{s}$
end for

Strict path: $D = X + H$ and full-coarse assets

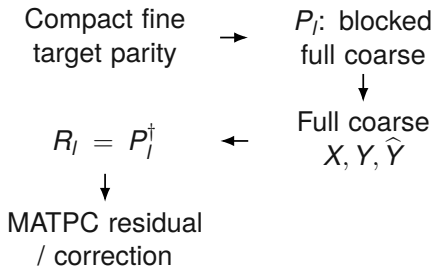
Level decomposition

$$\begin{aligned} D_l &= X_l + H_l \\ \hat{D}_l &= X_l^{-1} D_l \end{aligned} \quad (13)$$

$$D_{l+1} = R_l \hat{D}_l P_l$$

$$\mathcal{G}_l = \{V_l, X_{l+1}, X_{l+1}^{-1}, Y_{l+1}^{\text{f/b}}, \hat{Y}_{l+1}^{\text{f/b}}\} \quad (14)$$

Runtime assets: $X, X^{-1}, Y^{\text{f/b}}, \hat{Y}^{\text{f/b}}, V$. Coarse levels keep full geometry; fine level uses compact target parity.



Coarse X/Y : onsite, directional links and $\hat{Y}$

Construction of X and Y

$$E = 2n_v, \quad X_\ell, Y_{\ell,\mu}^{f/b} \in \mathbb{C}^{E \times E} \quad (15)$$

$$X_C(X) = \sum_{x \in \mathcal{A}_X} V_X^\dagger C V_X \quad (16)$$

$$Y_\mu^f(X) = -\kappa \sum_{x \in \mathcal{A}_X} (A V_X)^\dagger U_\mu V_{X+\hat{\mu}} \quad (17)$$

$$Y_\mu^b(X - \hat{\mu}) = -\kappa \sum_{x \in \mathcal{A}_X} V_X^\dagger U_\mu^\dagger A V_{X-\hat{\mu}} \quad (18)$$

$AV = C^{-1}V$ (Clover-PC); raw path 取 $AV = V$ 。

Raw action and preconditioned links

$$D_c z(X) = X(X)z(X) + \sum_{\mu} [Y_\mu^f(X)z(X + \hat{\mu}) + Y_\mu^b(X - \hat{\mu})^\dagger z(X - \hat{\mu})] \quad (19)$$

$$\hat{Y}_\mu^f(X) = X^{-1}(X) Y_\mu^f(X) \quad (20)$$

$$\hat{Y}_\mu^b(X - \hat{\mu}) = Y_\mu^b(X - \hat{\mu}) X^{-\dagger}(X) \quad (21)$$

$$M_p^c x_p = x_p - \hat{H}_{pq} \hat{H}_{qp} x_p \quad (22)$$

$$\hat{H}z = \sum_{\mu} [\hat{Y}_\mu^f(X)z(X + \hat{\mu}) + \hat{Y}_\mu^b(X - \hat{\mu})^\dagger z(X - \hat{\mu})] \quad (23)$$

PC dslash 只应用 $\hat{Y}$ hopping; backward 使用 $X - \hat{\mu}$ 的 storage block, 左右矩阵顺序不可交换。

V-cycle inside, flexible FGMRES outside

Algorithm 3 Recursive MG preconditioner

Input : $A_l, r_l, \{S_{\text{pre}}, S_{\text{post}}, P_l, R_l\}$
if $l = L - 1$ **then** $z_l \leftarrow A_l^{-1} r_l$
else
 $y_l \leftarrow S_{\text{pre}}^{\nu_1}(r_l)$
 $r_{l+1} \leftarrow R_l(r_l - A_l y_l)$
 $e_{l+1} \leftarrow M_{l+1}^{-1} r_{l+1}$
 $z_l \leftarrow y_l + P_l e_{l+1}$
 $z_l \leftarrow S_{\text{post}}^{\nu_2}(z_l)$
end if

$$x^{(k+1)} = x^{(k)} + z^{(k)} \quad (24)$$

$$z_l = S_{\text{post}}^{\nu_2} \left[S_{\text{pre}}^{\nu_1} r_l + P_l M_{l+1}^{-1} R_l (r_l - A_l S_{\text{pre}}^{\nu_1} r_l) \right] \quad (25)$$

Algorithm 4 Flexible FGMRES

$r_0 = b - Ax^{(0)}, v_1 = r_0 / \|r_0\|$
for $j = 1, \dots, m$
 $z_j = M^{-1} v_j; w = Az_j$
 for $i = 1, \dots, j$
 $h_{ij} = v_i^\dagger w; w = w - h_{ij} v_i$
 end for
 $h_{j+1,j} = \|w\|; v_{j+1} = w / h_{j+1,j}$
 apply Givens rotations to H_j
end for
 $x^{(k+1)} = x^{(k)} + Z_j y$

CUDA–C++ execution path: fine kernel, coarse latency, MPI overlap

Algorithm 5 Distributed PyQCU MG call path

Input : $b_p, A_l, \{V_l, X_l, Y_l, \hat{Y}_l\}, p$

1. partition: rank-local / send / inside / receive
2. fine Dslash: $1 \pm \gamma_\mu$ projection $\rightarrow SU(3)$ transport
3. coarse hop: X onsite + $\hat{Y}$ forward/backward links
4. V-cycle: pre-smooth, restrict, recurse, prolong, post-smooth
5. outer FGMRES: Arnoldi + frozen/updated coarse hierarchy
6. correction: receive halo, add remote correction, true residual

hot path	optimization
fine kernel	4 $\rightarrow$ 2 spin projection; $SU(3)$ reuse; xyzt coalescing
coarse apply	33-point stencil; device scalar; fused X - $\hat{Y}$ update
Krylov	fused dot/vector update; CUDA Graph; smallest-level solve
setup	batched reduced QR; colored Galerkin; workspace cap
MPI	32-slot coarse halo; nonblocking vector halo; cached link halo; remote correction
cache	persistent hierarchy; HDF5 P/V/X/Y; schema/hash validation

CUDA–C++ internals: persistent state, fused recurrences, overlap boundary

Algorithm 6 Strict fused execution schedule

- 1. compact fine parity: $b_p \mapsto b_p^{pc}$
- 2. MATPC: $x_p \leftarrow x_p - \hat{H}_{pq} \hat{H}_{qp} x_p$
- 3. V-cycle: MR pre $\rightarrow R_l \rightarrow$ coarse solve $\rightarrow P_l \rightarrow A_l r_l \rightarrow$ MR post
- 4. right FGMRES: $z_j = \mathcal{M}^{-1} v_j$, $w = M_p z_j$; Arnoldi; restart true residual
- 5. halo: local base $\rightarrow$ post lrecv/lsend $\rightarrow$ wait $\rightarrow$ unpack $\rightarrow$ remote correction
- 6. cache: blocked $P/V/X/Y$; HDF5 shape/dtype/hash validation

mechanism		implementation detail			
fine Dslash		$1 \pm \gamma_\mu$: 4→2 projection; $SU(3)$ reuse; <i>xyzt</i> coalescing			
coarse	latency graphs	device scalar; fused β/ρ and r/x ; adaptive dot reduction			
		segment replay for legacy kernels; cooperative coarsest solve; safe fall-back			
persistent state		preallocated	streams/workspaces;		
		hierarchy	arena;	FGMRES	
		workspace			
strict overlap		c64	nonblocking	vector	halo;
		c128 default off; link halo remains			
		Sendrecv			
setup/cache		colored/batched		Galerkin;	
		workspace cap;	SHA256;	atomic	
		HDF5 publish			

QUDA comparison protocol: 66 exact units, 132 side records

device	precision	solvers	tracing	timing protocol
V100 / P100×2	c64 / c128	BiCGStab / MG-2 / MG-3	on / off	1 cold + 2 warmup + 5 steady

66 distinct exact/effective combined units · 132 side records · 6 extra capacity substitutions

$R = t_{\text{QUDA,steady}} / t_{\text{PyQCU,steady}}$ · trace-off is formal · test27 follows Strict MATPC + fused right-FGMRES

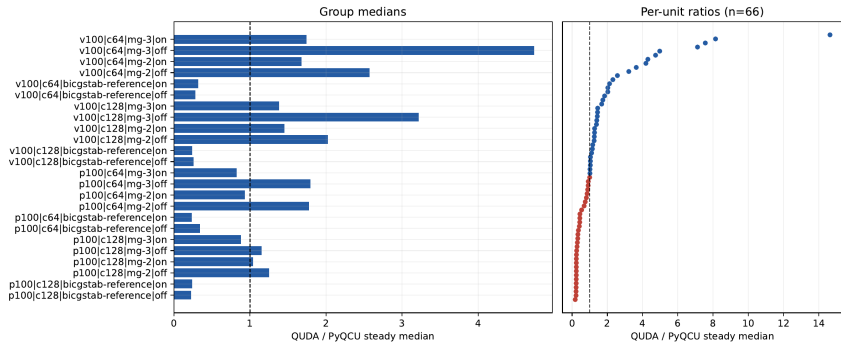


图 1：协议与 R 分布；替代项只作为容量与趋势证据，不进入原请求比值。

Overall result: MG-specific gain, total median 1.025

$1.025 = \text{median}_{66} R \cdot 35/44 \text{ MG-2/3 with } R > 1 \text{ (on/off)} \cdot 21/22 \text{ trace-off MG} \cdot 0/22 \text{ BiCGStab} \cdot 0.1766\text{--}14.63 \text{ all-unit range}$

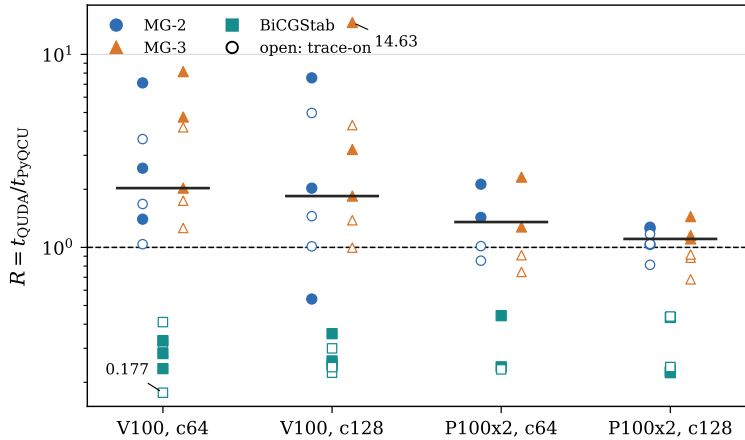


图 2：每个点是一个精确 unit，水平线为对应 trace-off 分组中位数。

Device and precision trends: V100 strong, P100 moderate

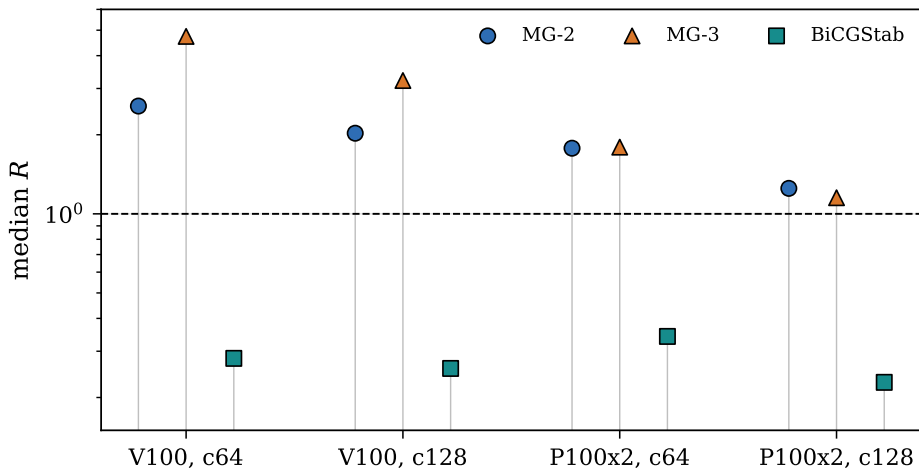


图 3: V100 c64/c128 MG-3 trace-off group medians $4.74 \times / 3.22 \times$; P100 $\times$ 2 c64/c128 MG-2 $1.78 \times / 1.25 \times$ 。P100 c64 每组仅 $n = 2$ 。

trace-on stage attribution: coarse + other dominate

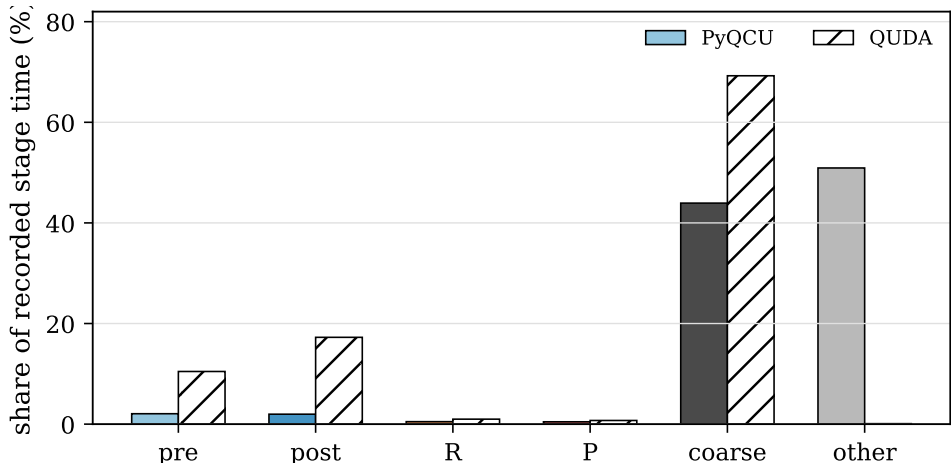
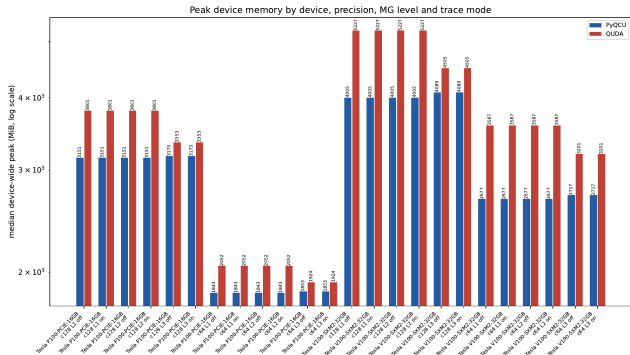


图 4: 逐 unit、逐侧阶段份额的中位数; PyQCU coarse + other $\approx 94.9\%$, QUDA 具名 coarse $\approx 69.3\%$ 。该量不是 wall-clock 总时间求和份额。

device-wide memory and correctness gate



Limitations and optimization roadmap

limitation	evidence	next action
coarse latency	PyQCU coarse + other $\approx 94.9\%$	fuse coarse solve, reduction and synchronization
setup cost	6 P100 c64 large requests substituted	colored/batched Galerkin; workspace streaming
BiCGStab	0/22 favorable	keep as reference; focus MG path
communication overlap	vector requests posted after local base kernel; link halo blocking	post requests earlier; overlap remote correction
device capacity	P100 c64 large requests exceed limits	staged halo; compressed assets; cache reuse
portability	CUDA verified; DCU/CANN not end-to-end	device-aware MPI and domestic-platform gates

$$T_{\text{total}} = T_{\text{fine}} + T_{\text{coarse}} + T_{\text{comm}} + T_{\text{setup}} \tag{26}$$

当前可证实的优势：MG 专项性能、V100、较低最大显存、扁平代码结构和自主动手能力。

当前不可宣称：任意格点、任意层数、所有 solver 或大规模 multi-rank 普遍领先。

下一步验收：每个优化都必须同时通过 true residual、iteration count、steady time 和 device-wide memory 四项门限。

Reference implementations and minimal reproduction

Reference implementations

- quantum-mg: Evan Weinberg / NVIDIA
- QUDA: multigrid, staging, coarse operator
- DD- α AMG: Matthias Rottmann
- 本地源码: `refer/git-rep/**`

Reproduction order

1. `source ./env.sh`
2. `bash ./build.sh & bash ./install.sh`
3. `run_strict_fast.py --fail-fast`
4. `strict_mpi_solve_probe.py`
5. `bench_mg_matrix_full.py --list`

Platform status: CUDA verified; CPU partial;
DCU/CANN historical/simulated; TileLang 为 smoke;
历史大规模测试见下一页。

Provenance: QUDA 1.1.0; PyQUDA 0.10.54; test27 commit
04382801; V100 sm70; P100 sm60。

Prior results: 508 report (TEST8 c64 / TEST9 c128)

case	precision	lattice	ranks	Wilson	Clover
TEST8	complex64	$128^3 \times 256$	1024	$3.119 \times$	$7.193 \times$
TEST9	complex128	$128^3 \times 256$	1024	$2.211 \times$	$4.258 \times$

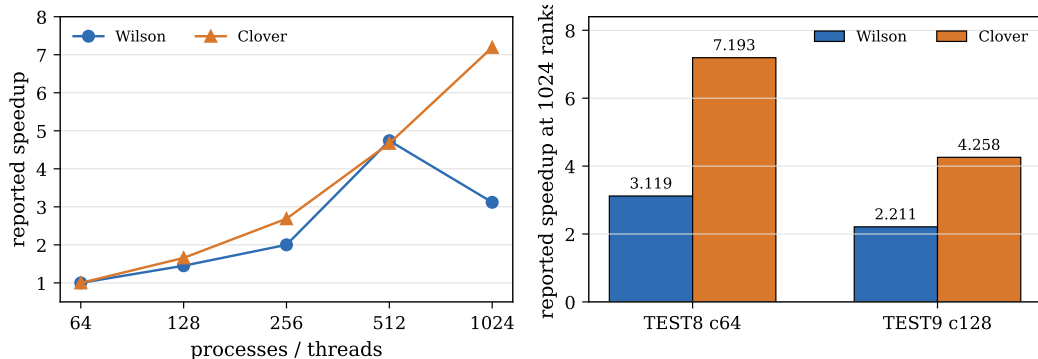


图 7: TEST8 c64 强扩展与 1024-rank 双精度对照。

64 卡池容量估算 ≈ 2048 GiB 来自 TEST8 原始表; 1024 是 MPI rank/thread 规模, 不是 1024 张 GPU 的实测聚合性能。

谢谢

PyQCU Strict MultiGrid

下一步：粗层同步、通信重叠和 setup 显存做成可复验收益。

`https://github.com/zhangxin8069/PyQCU`

数据基线: `gittagtest27`

Q & A